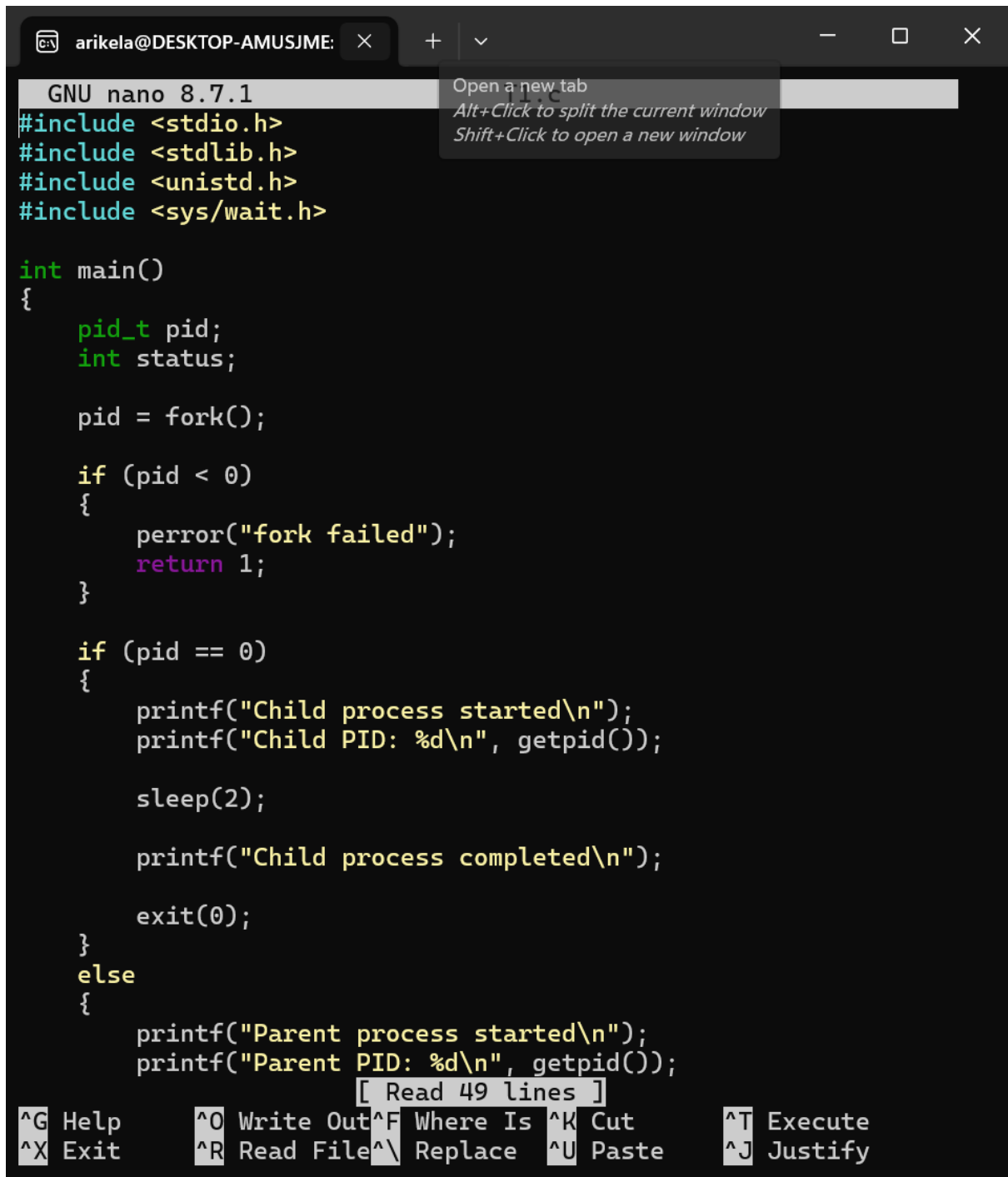


SKILL WEEK -4

TASK-1

Process Synchronization and Child Process Monitoring Using waitpid()

Sol:



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID: %d\n", getpid());

        sleep(2);

        printf("Child process completed\n");

        exit(0);
    }
    else
    {
        printf("Parent process started\n");
        printf("Parent PID: %d\n", getpid());

        [ Read 49 lines ]
    }
}
```

Open a new tab
Alt+Click to split the current window
Shift+Click to open a new window

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```
if (pid == 0)
{
    printf("Child process started\n");
    printf("Child PID: %d\n", getpid());

    sleep(2);

    printf("Child process completed\n");

    exit(0);
}
else
{
    printf("Parent process started\n");
    printf("Parent PID: %d\n", getpid());

    printf("Waiting for child process...\n");

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
               WEXITSTATUS(status));
    }

    printf("Parent process completed\n");
}

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
arikel@DESKTOP-AMUSJME: ~$ mkdir skill4
arikel@DESKTOP-AMUSJME: ~$ cd skill4
arikel@DESKTOP-AMUSJME: ~/skill4$ pwd
/home/arikel/skill4
arikel@DESKTOP-AMUSJME: ~/skill4$ nano T1.c
arikel@DESKTOP-AMUSJME: ~/skill4$ gcc T1.c -o T1
arikel@DESKTOP-AMUSJME: ~/skill4$ ./T1
Parent process started
Parent PID: 576
Waiting for child process...
Child process started
Child PID: 577
Child process completed
Child exited with status: 0
Parent process completed
```

TASK-2

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

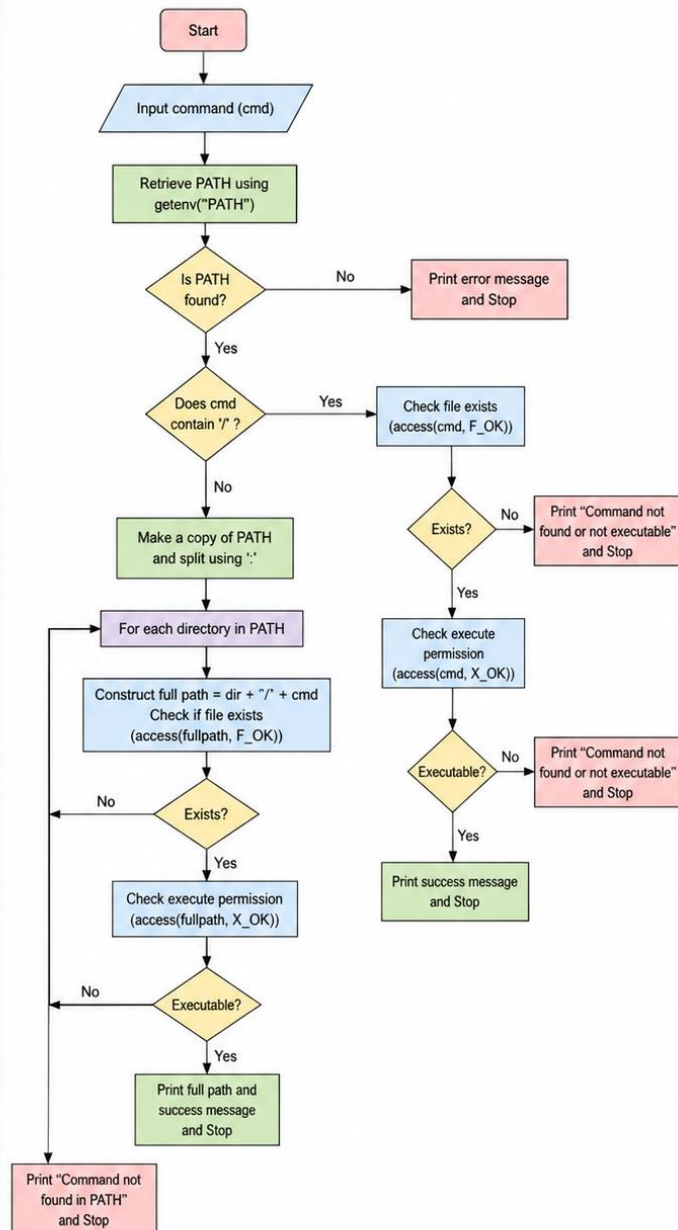
Function	Purpose
getenv()	Retrieves the PATH environment variable
strtok()	Splits PATH into individual directories
access()	Checks file existence and execute permission
snprintf()	Builds the complete executable path
strdup()	Creates a modifiable copy of PATH
free()	Releases allocated memory

Flow Chart:

ALGORITHM

1. Start
2. Input command name from user (cmd).
3. Retrieve the PATH environment variable using `getenv("PATH")`.
If PATH is not found, print error message and stop.
4. If cmd contains '/' (i.e., user gave a path):
 - 4.1. Check if the file exists using `access(cmd, F_OK)`.
 - 4.2. If it exists, check execute permission using `access(cmd, X_OK)`.
 - 4.3. If executable, print success message.
 - 4.4. Else, print "Command not found or not executable".
 - 4.5. Stop.
5. Else (cmd does not contain '/'):
 - 5.1. Make a copy of PATH.
 - 5.2. Split the PATH using ':' as delimiter to get directories.
 - 5.3. For each directory in PATH do
 - 5.3.1. Construct full path = directory + '/' + cmd.
 - 5.3.2. Check if file exists at full path using `access(fullpath, F_OK)`.
 - 5.3.3. If file exists, check execute permission using `access(fullpath, X_OK)`.
 - 5.3.4. If executable, print full path and success message. Go to Step 6.
 - 5.3.5. Else, continue with next directory.
 - 5.4. If all directories are checked and command not found, print "Command not found in PATH".
6. Stop.

FLOWCHART



Sample Output:

Case 1:

`./T2 pwd`

PATH =

`/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/mnt/c/Program`

`Files/WindowsApps/CanonicalGroupLimited.Ubuntu_2404.1.68.0_x64__79rhkp1fndgsc:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/Microsoft VS`

Code/bin:/mnt/c/Users/DELL/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/DELL/AppData/Local/GitHubDesktop/bin:/snap/bin

File found: /usr/bin/pwd

Execute permission: YES

Command resolved successfully.

Case 2:

./T2 Raghu

PATH =

/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/mnt/c/Program

Files/WindowsApps/CanonicalGroupLimited.Ubuntu_2404.1.68.0_x64__79rhkp1fndgsc:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/Microsoft VS

Code/bin:/mnt/c/Users/DELL/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/DELL/AppData/Local/GitHubDesktop/bin:/snap/bin

Command 'Raghu' not found in PATH.

GNU nano 8.7.1

T2.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    char *path;
    char *path_copy;
    char *dir;
    char fullpath[1024];

    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n\n", path);

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        perror("strdup failed");
    }
}
```

^G Help

^O Write Out

^F Where Is

^K Cut

^T Execute

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

```
if (path == NULL)
{
    printf("PATH variable not found.\n");
    return 1;
}

printf("PATH = %s\n\n", path);

path_copy = strdup(path);

if (path_copy == NULL)
{
    perror("strdup failed");
    return 1;
}

dir = strtok(path_copy, ":");

while (dir != NULL)
{
    snprintf(fullpath, sizeof(fullpath),
             "%s/%s", dir, argv[1]);

    if (access(fullpath, F_OK) == 0)
    {
        printf("File found: %s\n", fullpath);

        if (access(fullpath, X_OK) == 0)
        {
            printf("Execute permission: YES\n");
            printf("Command resolved successfully.\n");

            free(path_copy);
        }
    }
}
```

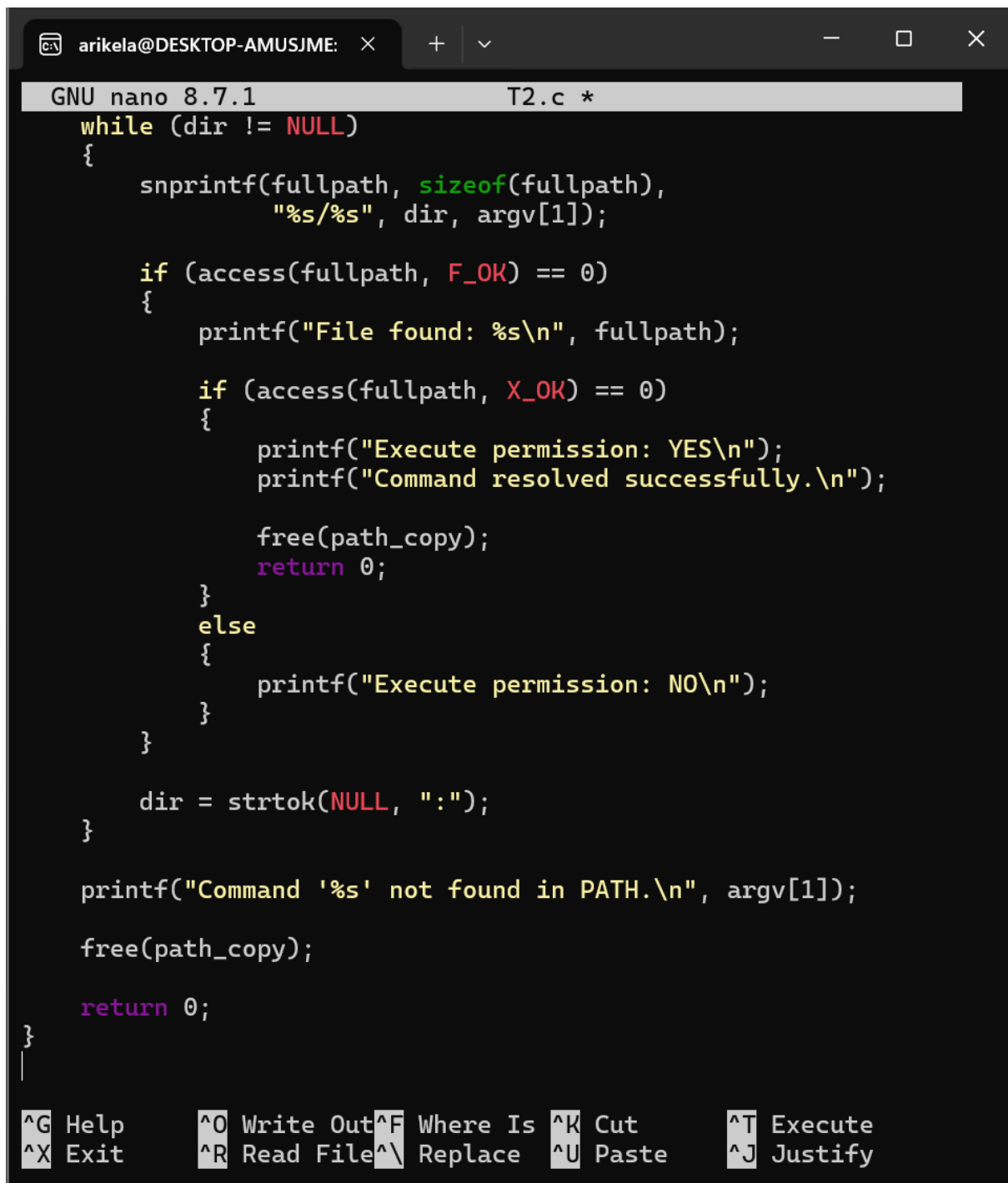
^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify



```
GNU nano 8.7.1 T2.c *
while (dir != NULL)
{
    snprintf(fullpath, sizeof(fullpath),
             "%s/%s", dir, argv[1]);

    if (access(fullpath, F_OK) == 0)
    {
        printf("File found: %s\n", fullpath);

        if (access(fullpath, X_OK) == 0)
        {
            printf("Execute permission: YES\n");
            printf("Command resolved successfully.\n");

            free(path_copy);
            return 0;
        }
        else
        {
            printf("Execute permission: NO\n");
        }
    }

    dir = strtok(NULL, ":");
}

printf("Command '%s' not found in PATH.\n", argv[1]);

free(path_copy);

return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

Case-1:


```
Parent process completed
arikel@DESKTOP-AMUSJME:~/skill4$ nano T1.c
arikel@DESKTOP-AMUSJME:~/skill4$ nano T2.c
arikel@DESKTOP-AMUSJME:~/skill4$ gcc T2.c -o T2
arikel@DESKTOP-AMUSJME:~/skill4$ ./T2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program
Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinu
x_2.7.10.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Ora
cle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Wi
ndows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1
.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/dotnet
:/mnt/c/MinGW/bin:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/P
rogram Files/nodejs:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Pr
ogram Files/Git/cmd:/mnt/c/Users/arikel/.local/bin:/mnt/c/Program
Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/arikel/AppData/Loc
al/Microsoft/WindowsApps:/mnt/c/Users/arikel/AppData/Local/Progra
ms/Microsoft VS Code/bin:/mnt/d/cprog/S10C/bin:/mnt/c/Users/arike
l/AppData/Local/GitHubDesktop/bin:/mnt/c/Users/arikel/AppData/Roa
ming/npm:/mnt/c/Users/arikel/AppData/Local/Python/bin:/snap/bin

File found: /usr/bin/pwd
Execute permission: YES
Command resolved successfully.
arikel@DESKTOP-AMUSJME:~/skill4$ ./T2 Raghu
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program
Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinu
x_2.7.10.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Ora
cle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Wi
ndows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1
.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/dotnet
:/mnt/c/MinGW/bin:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/P
```

Case-2:

```

arikel@DESKTOP-AMUSJME:~/skill4$ ./T2 Raghu
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/
bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program
Files/WindowsApps/MicrosoftCorporationII.WindowsSubsystemForLinu
x_2.7.10.0_x64__8wekyb3d8bbwe:/mnt/c/Program Files/Common Files/Ora
cle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracl
e/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mnt/c/Wi
ndows/System32/Wbem:/mnt/c/Windows/System32/WindowsPowerShell/v1
.0:/mnt/c/Windows/System32/OpenSSH:/mnt/c/Program Files/dotnet
:/mnt/c/MinGW/bin:/mnt/c/Program Files/Java/jdk-25/bin:/mnt/c/P
rogram Files/nodejs:/mnt/c/ProgramData/chocolatey/bin:/mnt/c/Pr
ogram Files/Git/cmd:/mnt/c/Users/arikel/.local/bin:/mnt/c/Program
Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/arikel/AppData/Loc
al/Microsoft/WindowsApps:/mnt/c/Users/arikel/AppData/Local/Progra
ms/Microsoft VS Code/bin:/mnt/d/cprog/S10C/bin:/mnt/c/Users/arike
/AppData/Local/GitHubDesktop/bin:/mnt/c/Users/arikel/AppData/Roa
ming/npm:/mnt/c/Users/arikel/AppData/Local/Python/bin:/snap/bin

Command 'Raghu' not found in PATH.
arikel@DESKTOP-AMUSJME:~/skill4$ |

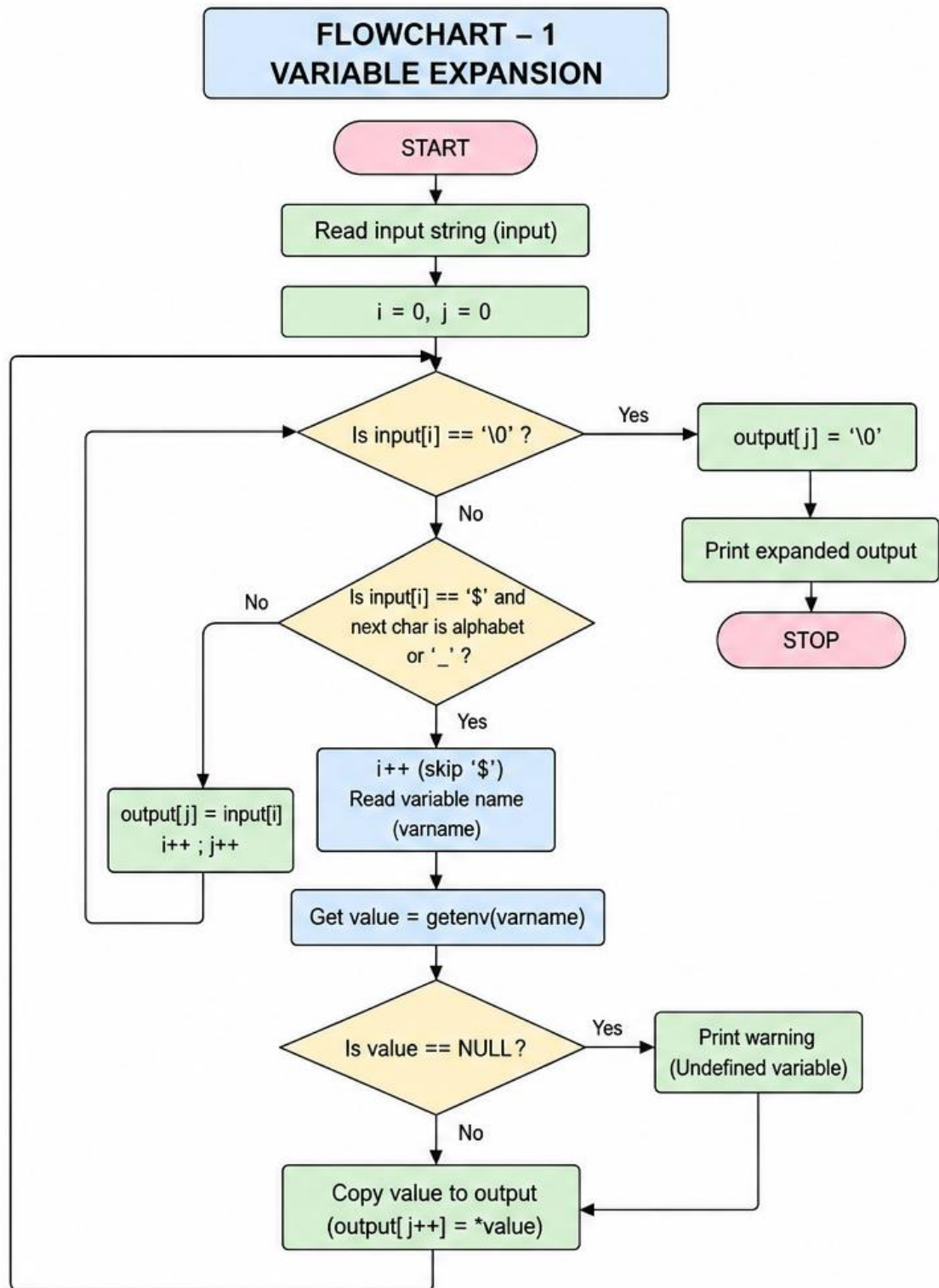
```

Task-3

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

Function	Purpose
getenv()	Retrieves environment-variable values
fgets()	Reads input
strcspn()	Removes newline
isalpha()	Checks alphabetic character
isalnum()	Checks alphanumeric character

Flow Chart:



Sample Output:

Case 1:

Enter a string: Raghu

Original : Raghu

Expanded : Raghu

Case 2:

Enter a string: hello \$abc

Warning: Undefined variable: abc

Original : hello \$abc

Expanded : hello

GNU nano 8.7.1

T3.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 1024

int main()
{
    char input[MAX];
    char output[MAX];

    int i = 0;
    int j = 0;

    printf("Enter a string: ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0' && j < MAX - 1)
    {
        if (input[i] == '$')
        {
            i++;

            char variable[100];
            int k = 0;

            if (isalpha(input[i]) || input[i] == '_')
            {
                while ((isalnum(input[i]) || input[i] == '_')
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^\ **Replace**

^K Cut
^U Paste

^T Execute
^J Justify

```
while ((isalnum(input[i]) || input[i] == '_')
      && k < 99)
{
    variable[k++] = input[i++];
}

variable[k] = '\0';

char *value = getenv(variable);

if (value != NULL)
{
    int len = strlen(value);

    if (j + len < MAX - 1)
    {
        strcpy(&output[j], value);
        j += len;
    }
}
else
{
    printf("Warning: Undefined variable: %s\n",
          variable);
}

}
else
{
    output[j++] = '$';
}
}
else
{

```

```
GNU nano 8.7.1 T3.c *
int len = strlen(value);

if (j + len < MAX - 1)
{
    strcpy(&output[j], value);
    j += len;
}
else
{
    printf("Warning: Undefined variable: %s\n",
        variable);
}
else
{
    output[j++] = '$';
}
else
{
    output[j++] = input[i++];
}

output[j] = '\0';

printf("Original : %s\n", input);
printf("Expanded : %s\n", output);

return 0;
}
|
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify

Case 1 and Case 2

```

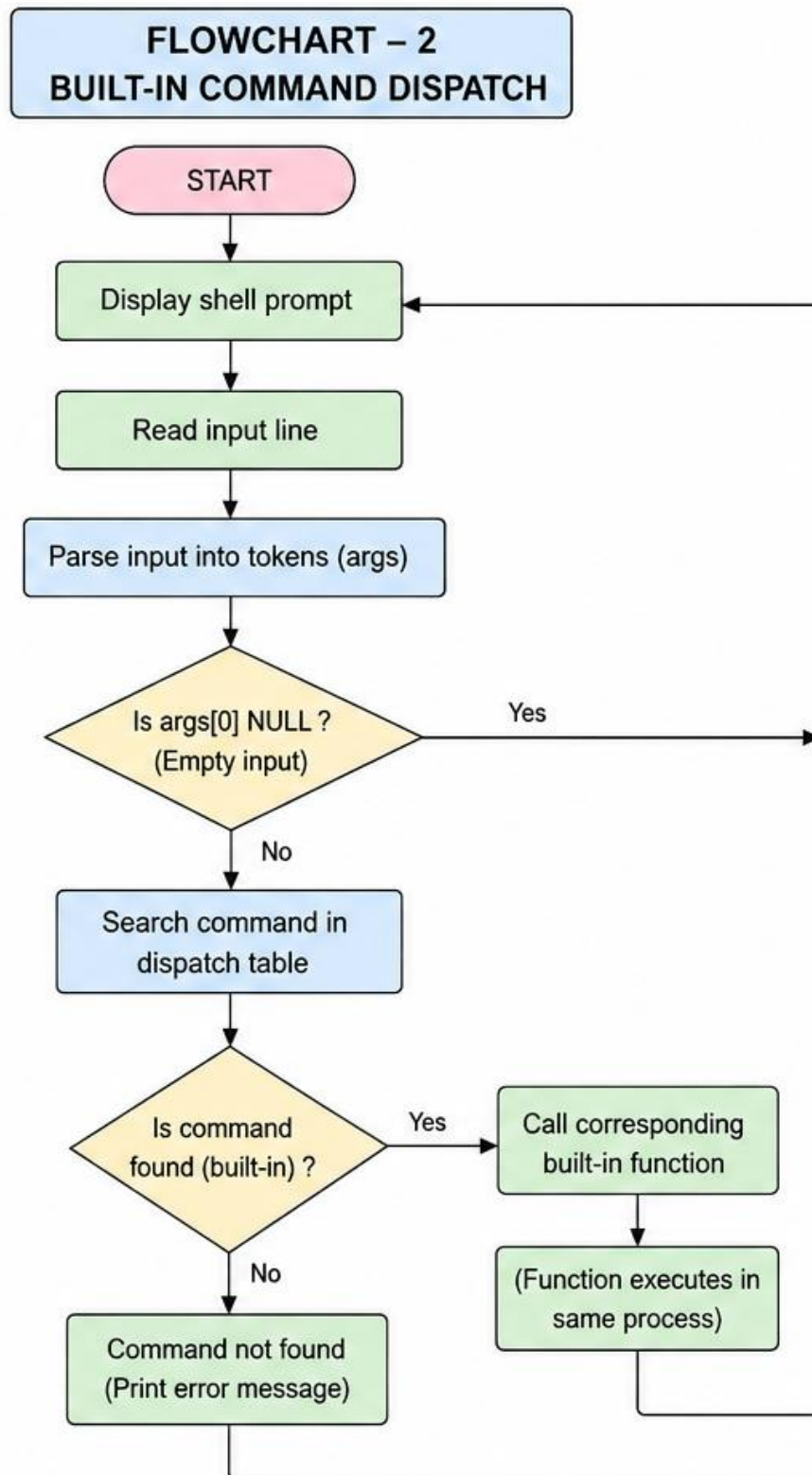
arikel@DESKTOP-AMUSJME:~/skill4$ nano T3.c
arikel@DESKTOP-AMUSJME:~/skill4$ gcc T3.c -o T3
arikel@DESKTOP-AMUSJME:~/skill4$ ./T3
Enter a string: Raghu
Original : Raghu
Expanded : Raghu
arikel@DESKTOP-AMUSJME:~/skill4$ ./T3
Enter a string: hello $abc
Warning: Undefined variable: abc
Original : hello $abc
Expanded : hello
arikel@DESKTOP-AMUSJME:~/skill4$ ./T3
Enter a string: hello $HOME
Original : hello $HOME
Expanded : hello /home/arikel
arikel@DESKTOP-AMUSJME:~/skill4$ |

```

Task 4: To Identify Built-ins, Create Dispatch Table, Execute In-Process Built-ins, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Function	Purpose
chdir()	Changes the current working directory
getcwd()	Gets the current working directory
getenv()	Retrieves environment variables
exit()	Terminates the process
strcmp()	Compares command names
strtok()	Separates command into tokens

Flow Chart:



Sample output:

myshell> pwd

/home/student

myshell> echo Hello World

Hello World

myshell> cd /tmp

myshell> pwd

/tmp

myshell> abc

Command not found: abc

myshell> exit

Exiting shell...

Sol:

arikel@DESKTOP-AMUSJME: x

+

▼

—

□

×

GNU nano 8.7.1

task4.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

// Built-in function declarations
void builtin_pwd(char **args);
void builtin_cd(char **args);
void builtin_echo(char **args);
void builtin_exit(char **args);

// Structure for dispatch table
struct builtin {
    char *name;
    void (*function)(char **args);
};

// Dispatch table
struct builtin builtins[] = {
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"echo", builtin_echo},
    {"exit", builtin_exit}
};

int builtin_count = 4;

// pwd command
void builtin_pwd(char **args)
{
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
// pwd command
void builtin_pwd(char **args)
{
    char cwd[MAX_INPUT];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

// cd command
void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        char *home = getenv("HOME");

        if (home != NULL)
            chdir(home);
        else
            perror("cd");
    }
    else
    {
        if (chdir(args[1]) != 0)
            perror("cd");
    }
}

// echo command
void builtin_echo(char **args)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```
// echo command
void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");
}

// exit command
void builtin_exit(char **args)
{
    printf("Exiting shell...\n");
    exit(0);
}

// Find and execute built-in command
int execute_builtin(char **args)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(args[0], builtins[i].name) == 0)
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
// Find and execute built-in command
int execute_builtin(char **args)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(args[0], builtins[i].name) == 0)
        {
            builtins[i].function(args);
            return 1;
        }
    }

    return 0;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char *token;
    int argc;

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
fflush(stdout);

if (fgets(input, sizeof(input), stdin) == NULL)
    break;

// Remove newline
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
    continue;

// Tokenize input
argc = 0;
token = strtok(input, " ");

while (token != NULL && argc < MAX_ARGS - 1)
{
    args[argc++] = token;
    token = strtok(NULL, " ");
}

args[argc] = NULL;

// Execute built-in
if (!execute_builtin(args))
{
    printf("Command not found: %s\n", args[0]);
}

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^N Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
arikel@DESKTOP-AMUSJME: ~$ nano task4.c
arikel@DESKTOP-AMUSJME: ~$ gcc task4.c -o task4
arikel@DESKTOP-AMUSJME: ~$ ./task4
myshell> pwd
/home/arikel
myshell> echo Hello World
Hello World
myshell> cd /tmp
myshell> pwd
/tmp
myshell> abc
Command not found: abc
myshell> exit
Exiting shell...
arikel@DESKTOP-AMUSJME: ~$
```